

Task-1: Find the value of $x=1.17$ using Gauss Central Forward Interpolation:

x	1.00	1.05	1.10	1.15	1.20	1.25	1.30
$y=e^x$	2.7183	2.8577	3.0042	3.1582	3.3201	3.4903	3.6693

Code (Python):

```
from prettytable import PrettyTable
import math

value_x=[1.00,1.05,1.10,1.15,1.20,1.25,1.30]
value_y=[2.7183,2.8577,3.0042,3.1582,3.3201,3.4903,3.6693]
table = PrettyTable()

def create_table(n):
    field=["x","y","Δy"]
    for i in range(2,n+1):
        field.append(f"Δ{i}y")
    table.field_names=field
    table.float_format=".5"

def Gauss_Central_Forward(n):
    create_table(len(value_y)-1)
    del_value=[]
    temp=[]
    for i in range(1,len(value_y)):
        temp.append(value_y[i]-value_y[i-1])
    del_value.append(temp)

    for i in range(1,len(value_y)-1):
        temp=[]
        for j in range(1,len(del_value[i-1])):
            temp.append(del_value[i-1][j]-del_value[i-1][j-1])
        del_value.append(temp)

    for i in range(len(value_x)):
        row=[value_x[i],value_y[i]]
        for j in range(len(del_value)):
            if i+1>len(del_value[j]):
                row.append("")
                continue
            row.append(del_value[j][i])
        table.add_row(row=row)
```

```
mid=math.ceil(len(value_y)/2)-1
p=(n-value_x[mid])/abs(value_x[0]-value_x[1])
q=1
result=value_y[mid]
for i in range(0,len(del_value)):
    if i%2==0:
        q*=(p-i)
        m=int(len(del_value[i])/2)
    else:
        q*=(p+i)
        m=int(len(del_value[i])/2)
    result+=del_value[i][m]*q/math.factorial(i)

return result

val=1.17
result=Gauss_Central_Forward(val)
print(table)
print(f'y({val})={result:.5f}\nError: {abs(math.e**val-result):.5f}')
```

Output:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$	$\Delta^5 y$	$\Delta^6 y$
1.00000	2.71830	0.13940	0.00710	0.00040	0.00000	-0.00000	0.00010
1.05000	2.85770	0.14650	0.00750	0.00040	-0.00000	0.00010	
1.10000	3.00420	0.15400	0.00790	0.00040	0.00010		
1.15000	3.15820	0.16190	0.00830	0.00050			
1.20000	3.32010	0.17020	0.00880				
1.25000	3.49030	0.17900					
1.30000	3.66930						

y(1.17)=3.22730
Error:0.00531

Task-2: Find the value of $x=1.17$ using Gauss Central Backward Interpolation:

x	1.00	1.05	1.10	1.15	1.20	1.25	1.30
$y=e^x$	2.7183	2.8577	3.0042	3.1582	3.3201	3.4903	3.6693

Code (Python):

```
from prettytable import PrettyTable
import math

value_x=[1.00,1.05,1.10,1.15,1.20,1.25,1.30]
value_y=[2.7183,2.8577,3.0042,3.1582,3.3201,3.4903,3.6693]
table = PrettyTable()

def create_table(n):
    field=["x","y","Δy"]
    for i in range(2,n+1):
        field.append(f"Δ{i}y")
    table.field_names=field
    table.float_format=".5"

def Gauss_Central_Backward(n):
    create_table(len(value_y)-1)
    del_value=[]
    temp=[]
    for i in range(1,len(value_y)):
        temp.append(value_y[i]-value_y[i-1])
        del_value.append(temp)

    for i in range(1,len(value_y)-1):
        temp=[]
        for j in range(1,len(del_value[i-1])):
            temp.append(del_value[i-1][j]-del_value[i-1][j-1])
        del_value.append(temp)

    for i in range(len(value_x)):
        row=[value_x[i],value_y[i]]
        for j in range(len(del_value)):
            if i+1>len(del_value[j]):
                row.append("")
                continue
            row.append(del_value[j][i])
        table.add_row(row=row)
```

```

mid=math.ceil(len(value_y)/2)-1
p=(n-value_x[mid])/abs(value_x[0]-value_x[1])
q=1
result=value_y[mid]
for i in range(0,len(del_value)):
    if i%2==0:
        q*=(p+i)
        m=int(len(del_value[i])/2)-1
    else:
        q*=(p-i)
        m=int(len(del_value[i])/2)
    result+=del_value[i][m]*q/math.factorial(i)

return result

val=1.17
result=Gauss_Central_Backward(val)
print(table)
print(f'y({val})={result:.5f}\nError: {abs(math.e**val-result):.5f}')

```

Output:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$	$\Delta^5 y$	$\Delta^6 y$
1.00000	2.71830	0.13940	0.00710	0.00040	0.00000	-0.00000	0.00010
1.05000	2.85770	0.14650	0.00750	0.00040	-0.00000	0.00010	
1.10000	3.00420	0.15400	0.00790	0.00040	0.00010		
1.15000	3.15820	0.16190	0.00830	0.00050			
1.20000	3.32010	0.17020	0.00880				
1.25000	3.49030	0.17900					
1.30000	3.66930						

$y(1.17)=3.21776$
 Error:0.00423

Task-3: Find the value of $x=2.7$ from the following table using Lagrange Interpolation:

x	2	2.5	3
y=ln(x)	0.69315	0.91629	1.09861

Code (Python):

```
import math
def lagranje(x,n):
    res=1.0
    A=upper_part(x,n)
    for i in range(len(arr_x)):
        if i==n:
            continue
        res*=(arr_x[n]-arr_x[i])
    res=(A/res)*arr_y[n]
    return res

def upper_part(x,n):
    res=1.0
    for i in range(len(arr_x)):
        if i==n:
            continue
        res*=(x-arr_x[i])
    return res

arr_x=[]
arr_y=[]
n=int(input())
for i in range(n):
    x,y=map(float,input().split())
    arr_x.append(x)
    arr_y.append(y)

val=float(input())
final=0.0
for i in range(n):
    final+=lagranje(val,i)
print(f' {final:.5f} ')
print(f'Error: {abs(final-math.log(val)):.5f}')
```

Output:

```
3
2 0.69315
2.5 0.91629
3 1.09861
2.7
y(2.7)=0.99412
Error: 0.00086
```

Discussion:

Interpolation is a mathematical technique used to estimate the value of a function for a given point within the range of known data points. In simple terms, it involves finding a value between two known values in a dataset. In this lab report I have coded 3 types of interpolation formula, a) Gauss Central Forward Interpolation, b) Gauss Central Backward Interpolation, c) Lagrange Interpolation.

For Gauss Central Interpolations, we must make the input values a series of consecutive numbers but in Lagrange we don't need to maintain any sequence. Gauss is useful for equal intervals and Lagrange is used for unequal intervals.